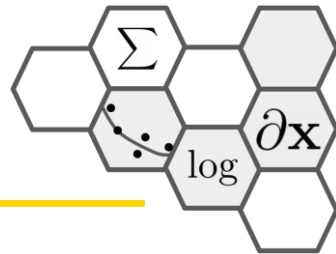
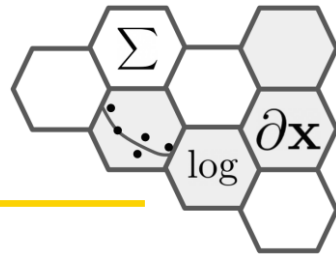


넘파이|Numpy



- Numpy is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays. If you are already familiar with MATLAB[®], you might find this tutorial useful to get started with Numpy. – cs231n

Install & Import

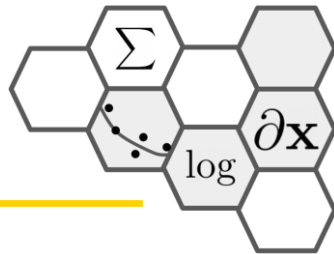


- 약칭을 np로 쓰는 것이 거의 표준

pip install numpy

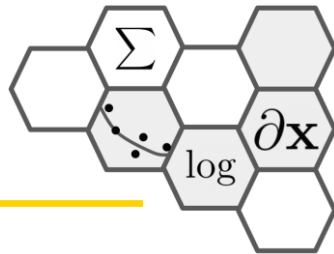
import numpy **as** np

What & Why



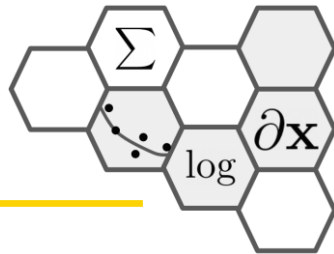
- NumPy 이란?
 - Python에서 과학 계산을 위한 핵심 라이브러리.
 - 고성능 다차원 배열 객체(ndarray)와 이를 다루는 도구들을 제공.
- 기본 리스트(List) 대신 NumPy를 쓰는 장점
 - 속도: NumPy 배열은 C로 구현된 저수준에서 최적화 (특히 대규모 데이터 연산 시)
 - 메모리 효율: 동일한 타입의 데이터를 연속된 메모리 공간에 저장하여 효율적
 - 편의성 (벡터화 연산): for 루프 없이 배열 전체에 대한 복잡한 수학/선형대수 연산이 가능
 - MATLAB의 행렬/배열 연산과 매우 유사

ndarray



- numpy에서 제공하는 메인 객체인 배열
- 다차원 배열 multidimensional array로써 1차원 배열(벡터), 2차원 배열(행렬), 3차원 배열(큐브), 4차원 배열(큐브가 여러 개 모인 것) 등등 계속 차원을 늘릴 수 있음

ndarray 생성



- 생성 : 행 우선 방식으로 []로 적어주면 됨
- Python 리스트와 만드는 방식 동일

```
import numpy as np
```

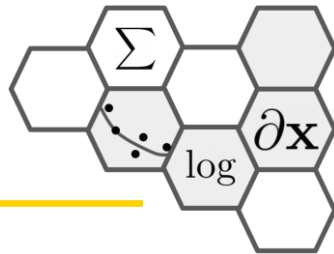
```
# 값을 지정하여 생성
```

```
A = np.array([[1,2],[3,4]])
```

순차적으로 다음 행을 나열

가로(행)으로 먼저 숫자를 나열하고

ndarray 생성



- 다양한 생성 방법

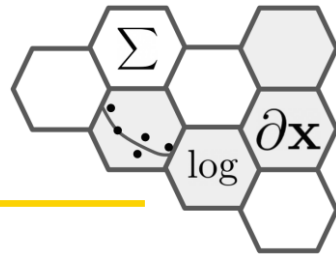
모든 요소가 0인 어레이
`B = np.zeros(3,3)`

모든 요소가 1인 어레이
`C = np.ones(2,3)`

대각 요소만 1인 어레이
`D = np.eye(2)`

랜덤 생성
`E = np.random.rand(10)`

Numpy 어레이 연산



- 기본연산: 덧셈, 뺄셈, 곱셈, 나눗셈은 요소끼리 계산
- Numpy 어레이 곱 $\neq$ 행렬곱

```
c = np.ones((3, 3))  
C = np.matrix(c)
```

```
print("numpy array multiplication")  
print(c*c)
```

```
[[1. 1. 1.]  
 [1. 1. 1.]  
 [1. 1. 1.]]
```

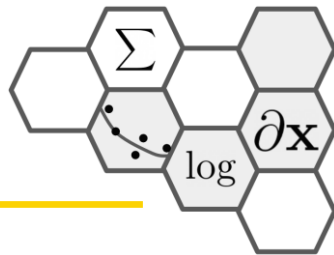
```
print("numpy dot function")  
print(c.dot(c))
```

```
[[3. 3. 3.]
```

```
print("numpy matrix multiplication")  
print(C*C)
```

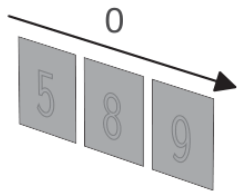
```
 [3. 3. 3.]  
 [3. 3. 3.]]
```

축 axis

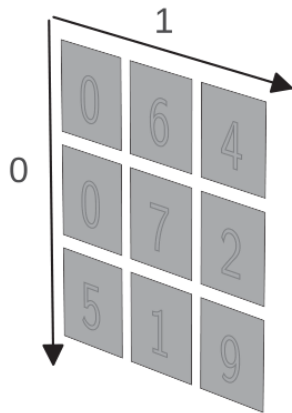


- 배열의 요소가 늘어선 방향
- 축에 번호를 붙여서 관리하는데 0부터 번호가 증가, 새롭게 생긴 axis가 0번

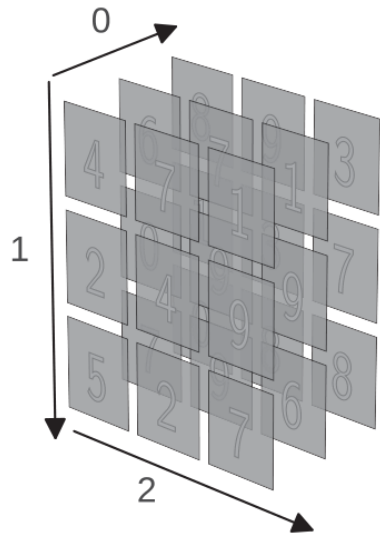
1D tensor: vector



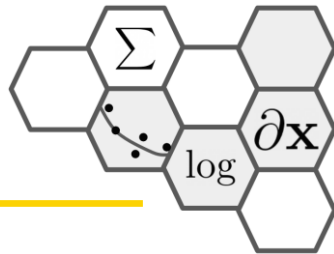
2D tensor: matrix



3D tensor



shape



- 배열의 모양

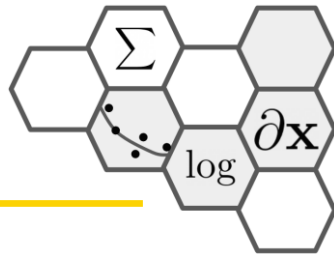
```
x = np.array([1,2,3])  
print(x.shape)    # (3,)
```

기본적으로 숫자 3개가 나열 됨
1차원 어레이

```
x = np.array([[1,2,3], [1,2,3]])  
print(x.shape)    # (2,3)
```

0번 축으로 2개, 1번 축으로 3개가 나열됨

reshape



- 기본 1차원 배열

```
x = np.array([1,2,3])  
print(x.shape) # (3,)
```

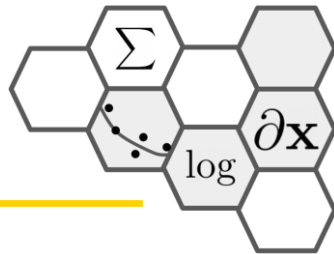
숫자가 가로로 나열되게

```
x_row_vector = x.reshape(1,3)  
print(x_row_vector)  
x_row_vector.shape
```

숫자가 세로로 나열되게

```
x_col_vector1 = x.reshape(3,1)  
print(x_col_vector1)  
print(x_col_vector1.shape)
```

reshape



- 2차원 이상에서 배열의 모양 변경

```
A = np.arange(12).reshape(4,3)
```

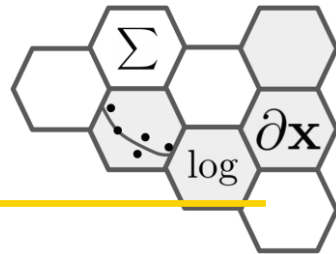
차원추가

```
A_ = A.reshape(-1, 2, 3)  
# (2,2,3)
```

차원축소

```
A_.reshape(-1,3)  
# (4,3)
```

자주쓰는 기능: arange



- Return evenly spaced values within a given interval.

start end

`np.arange(0, 10)`

`np.arange(1, 20, 2)`

step

A diagram showing a hexagonal lattice structure. A central hexagon is shaded light gray and contains the symbol $\partial \mathbf{x}$. To its left, another hexagon contains the symbol Σ . Below the Σ hexagon, a path of four dots is connected by a curved line, with the label $\log$ positioned below it. A thick yellow horizontal line is drawn across the bottom of the diagram.

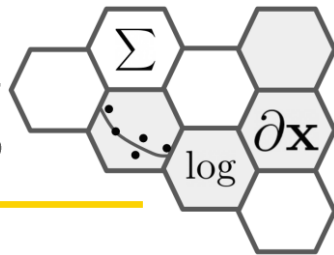
- Return evenly spaced numbers over a specified interval.

start ← end

np.linspace(0, 10, 5)

number

인덱싱indexing과 슬라이싱slicing



- 배열의 요소에 접근하기 위해서는 인덱스를 사용
- 인덱스는 0부터 시작, 끝 인덱스 포함 안됨
- 2축 이상에 대한 인덱싱
- 3행 3열 요소 $A[2,2]$
- $[start:end:stride]$
 - $A[:, 0::2]$: 행은 모든 행 $\rightarrow$:, 열은 start 0번부터, end 생략, stride는 2

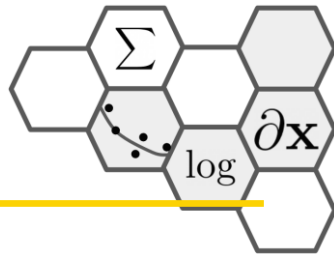
```
import numpy as np
```

```
A = np.arange(30).reshape(5,6)  
A
```

```
#>>> array([[ 0,  1,  2,  3,  4,  5],  
           [ 6,  7,  8,  9, 10, 11],  
           [12, 13, 14, 15, 16, 17],  
           [18, 19, 20, 21, 22, 23],  
           [24, 25, 26, 27, 28, 29]])
```

```
array([[ 0,  2,  4],  
       [ 6,  8, 10],  
       [12, 14, 16],  
       [18, 20, 22],  
       [24, 26, 28]])
```

어레이 인덱싱array indexing



- 인덱싱할 숫자를 임의의 어레이로 전달하는 방법
- 숫자를 지정할 자리에 숫자 대신 숫자 여러 개를 포함한 어레이를 전달하면 해당되는 요소 여러 개가 한번에 추출

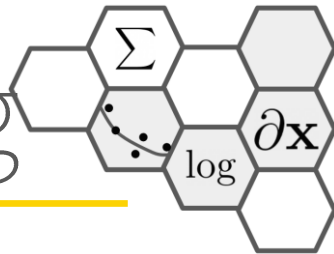
`np.array([A[0,1], A[1,2], A[2,3], A[3,1], A[4,0]])` → 추출할 모든 요소를 다 지정

```
#>>> array([ 1,  8, 15, 19, 24])
```

`A[[0,1,2,3,4], [1,2,3,1,0]]` → 추출할 모든 인덱스를 행과 열로 묶어 어레이로 지정

```
#>>> array([ 1,  8, 15, 19, 24])
```

불린 인덱싱 Boolean indexing



- Boolean 형 어레이를 전달

$A > 20$

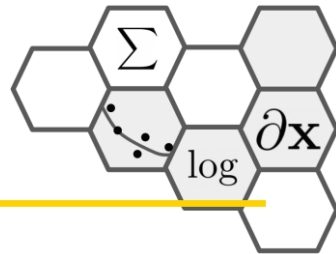
```
#>>> array([[False, False, False, False, False, False],  
            [False, False, False, False, False, False],  
            [False, False, False, False, False, False],  
            [False, False, False, True, True, True],  
            [ True,  True,  True,  True,  True,  True]])
```

$A[A > 20]$

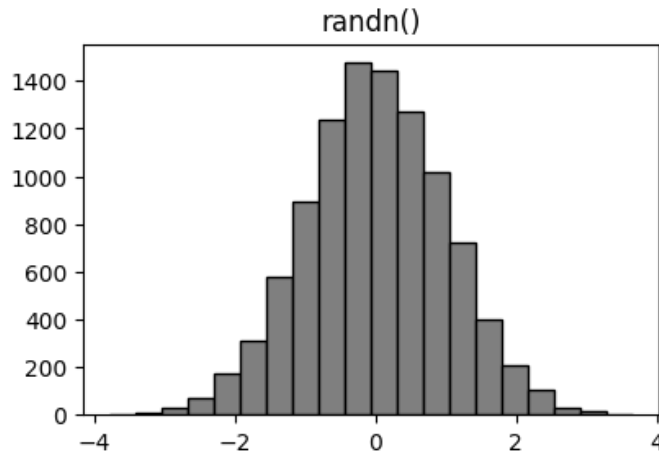
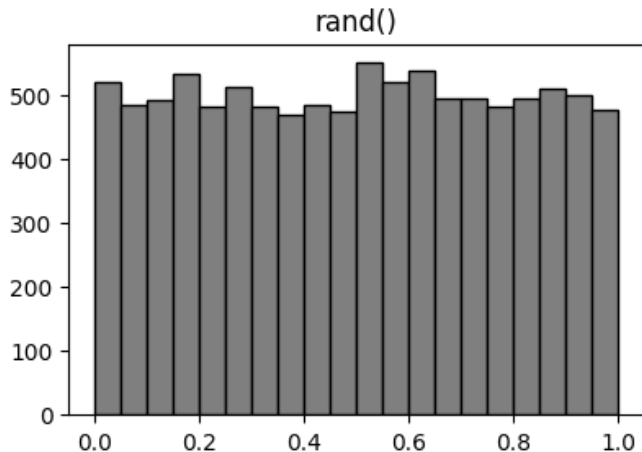
```
#>>> array([21, 22, 23, 24, 25, 26, 27, 28, 29])
```

20보다 큰 요소만 추출

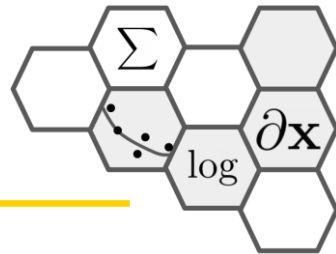
자주쓰는 기능: rand



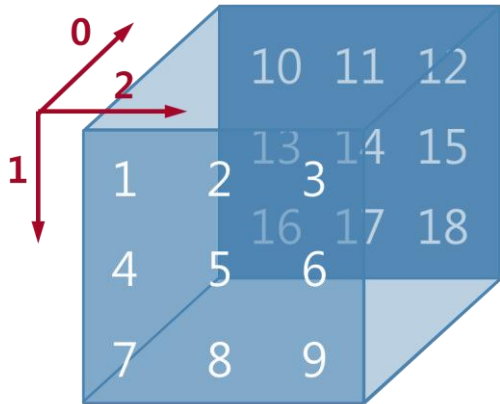
- `rand(): np.random.rand(10000)`
 - Create an array of the given shape and populate it with random samples from a uniform distribution over $[0, 1)$.
- `randn(): np.random.randn(10000)`
 - Return a sample (or samples) from the “standard normal” distribution.



전치|transpose



- `transpose(axis)` : 행렬의 전치와 같은 역할, 3개축 이상에서도 같은 논리로 동작



```
B = A[0,:]
```

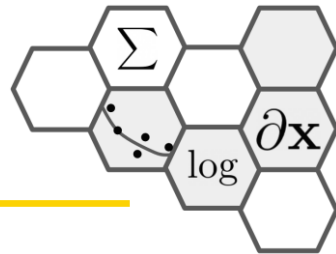
```
print(B)
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

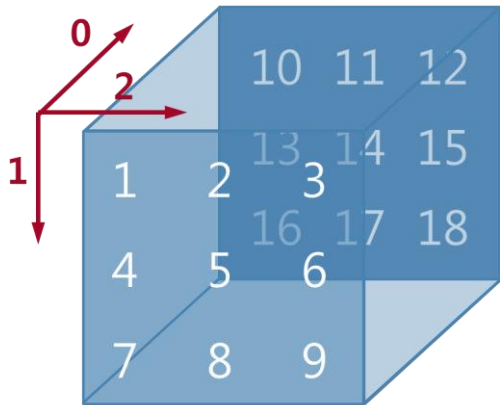
```
print(B.T)
```

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

전치|transpose



- `transpose(axis)` : 행렬의 전치와 같은 역할, 3개축 이상에서도 같은 논리로 동작



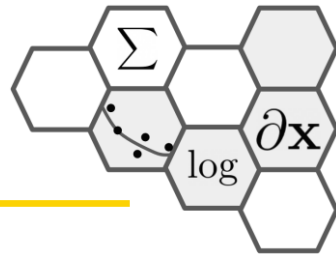
```
print(A.transpose(2, 0, 1))
```

```
[[[ 1  4  7]  
  [10 13 16]]
```

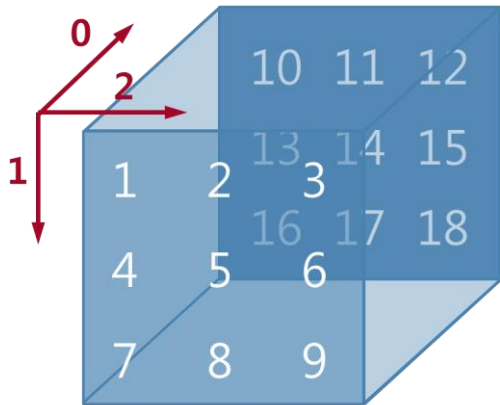
```
[[ 2  5  8]  
 [11 14 17]]
```

```
[[ 3  6  9]  
 [12 15 18]]]
```

전치|transpose



- `transpose(axis)` : 행렬의 전치와 같은 역할, 3개축 이상에서도 같은 논리로 동작



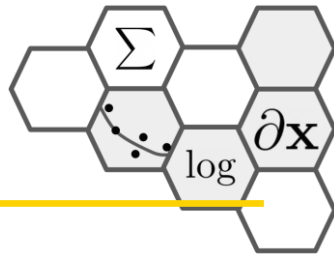
`print(?)`

```
[[[ 1, 10],  
 [ 2, 11],  
 [ 3, 12]],
```

```
[[ 4, 13],  
 [ 5, 14],  
 [ 6, 15]],
```

```
[[ 7, 16],  
 [ 8, 17],  
 [ 9, 18]]]
```

자주쓰는 기능



- `max()`
 - Return the maximum along a given axis.
- `sum()`, `mean()`, `std()`, `min()`, `argmin()`, `argmax()`
 - 모두 `max()` 와 동일하게 `axis`를 지정하여 연산

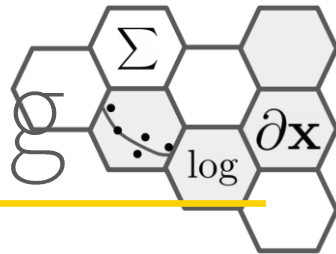
```
[[0.1891 0.1962 0.5885 0.6224 0.0469]
 [0.8457 0.8005 0.1416 0.9693 0.3967]] [0.6224 0.9693]
[0.8457 0.8005 0.5885 0.9693 0.3967]
```

`np.max(x)`
`np.argmax(x)`

`np.max(x, axis=0)`
`np.argmax(x, axis=0)`

`np.max(x, axis=1)`
`np.argmax(x, axis=1)`

브로드캐스팅broadcasting

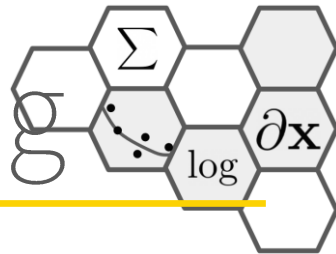


- Numpy에서 shape가 다른 배열 간에도 산술 연산이 가능하게 하는 메커니즘

The diagram shows three 3D arrays represented as boxes. The first array has values 1, 2, 3 and is labeled with shape (3,) below it. The second array has values 2, 2, 2 and is labeled with shape (1,) below it. The third array has values 2, 4, 6 and is labeled with shape (3,) below it. The equation is: (3,) array × (1,) array = (3,) array.

$$\begin{matrix} \begin{matrix} 1 & 2 & 3 \end{matrix} \\ (3,) \end{matrix} \times \begin{matrix} \begin{matrix} 2 & 2 & 2 \end{matrix} \\ (1,) \end{matrix} = \begin{matrix} \begin{matrix} 2 & 4 & 6 \end{matrix} \\ (3,) \end{matrix}$$

브로드캐스팅broadcasting



8	8	6
2	8	7
2	1	5
4	4	5

 $\times$

7	3	6
7	3	6
7	3	6
7	3	6

 $=$

56	24	36
14	24	42
14	3	30
28	12	30

(4,3)

(4,3)

(4,3)

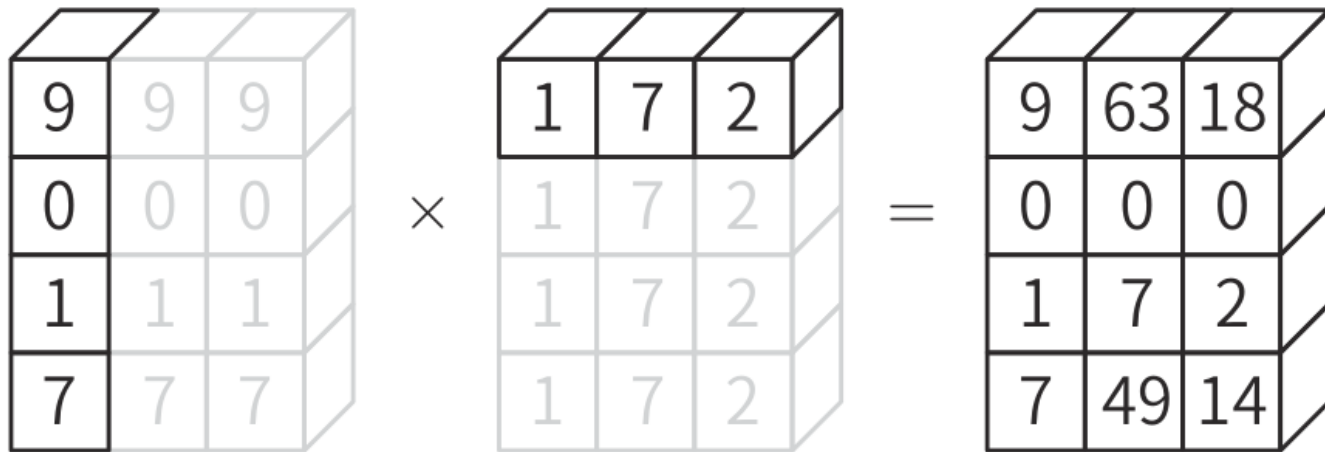
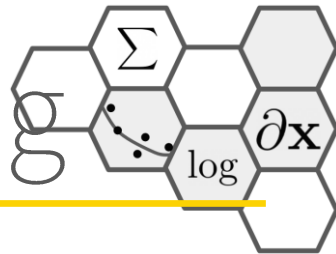
(3,)

(1,3)

(4,3)

(4,3)

브로드캐스팅broadcasting



(4,1)

(3,)

(4,1)

(1,3)

(4,3)

(4,3)

(4,3)